

Hoja de ruta del día

- **Módulo:** Módulo III: Machine Learning
- **Semana / Sesión:** Semana 14 - Fase 1
- **Temática Central:** Pipelines en Scikit-Learn: encapsulamiento de preprocesamiento y modelo para evitar data leakage
- **Stack Requerido:** Python, Scikit-Learn, Pandas, NumPy, Google Colab

Objetivos de aprendizaje del día

Al finalizar la jornada de investigación autónoma, debes ser capaz de:

- **Objetivo 1:** Comprender qué es el data leakage y por qué ocurre cuando el preprocesamiento se aplica sobre el dataset completo antes de dividirlo en train/test.
- **Objetivo 2:** Construir un **Pipeline** de Scikit-Learn que encadene al menos un transformador y un estimador, entendiendo la diferencia entre **fit()**, **transform()** y **fit_transform()** en cada etapa.
- **Objetivo 3:** Evaluar el impacto real del data leakage comparando métricas de un modelo con y sin Pipeline sobre el mismo dataset.

Bibliografía

Deberás investigar de forma independiente las siguientes fuentes de referencia para solucionar el reto técnico.

- **Documentación Oficial 1:** Scikit-Learn – [Pipeline: Chaining estimators](#) — leer especialmente cómo fluyen **fit** y **transform** entre etapas.
- **Documentación Oficial 2:** Scikit-Learn – [Common pitfalls: Data leakage](#) — sección específica con ejemplos de código incorrecto vs correcto.
- **Tutorial** complementario (YouTube):
 - [Build ML Pipelines with Scikit-Learn | Beginner Friendly](#)
 - [Preventing Data Leakage.](#)

Ejemplo de implementación práctica

El siguiente fragmento muestra el patrón correcto. **No resuelve el reto**: solo ilustra la sintaxis básica.

Python

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split

X, y = load_iris(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42)

# Patrón: el split SIEMPRE va antes que cualquier transformación
pipe = Pipeline(steps=[
    ('scaler', StandardScaler()),
    ('modelo', LogisticRegression())
])

pipe.fit(X_train, y_train)
print(f"Accuracy: {pipe.score(X_test, y_test):.4f}")
```

💡 Fíjate en el orden: primero **train_test_split**, luego el Pipeline. ¿Qué pasaría si se invierte ese orden y se escala **antes** del split?

Reto práctico de código:

Contexto del problema: Trabajas en el equipo de ML de DataScope Solutions. Una compañera ha entrenado un clasificador de especies de flores (Iris setosa, versicolor, virginica) y reporta una accuracy del 98%. Al revisar su código, encuentras esto:

Python

```
from sklearn.preprocessing import StandardScaler
from sklearn.impute import SimpleImputer

imputer = SimpleImputer(strategy='mean')
X_imputed = imputer.fit_transform(X) # ⚠️

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X_imputed) # ⚠️

X_train, X_test, y_train, y_test = train_test_split(X_scaled, y,
                                                    test_size=0.2)
```

Componente de incertidumbre:

Tu misión

1. *Identificar y explicar* en una celda Markdown: ¿en qué líneas ocurre el leakage y por qué es un problema aunque el dataset Iris sea pequeño y "limpio"?
2. *Refactorizar* el código usando un *Pipeline* con *SimpleImputer* + *StandardScaler* + *LogisticRegression*, asegurando que el leakage quede eliminado.
3. *Comparar* la accuracy del modelo con leakage vs. sin leakage usando *cross_val_score* con 5 folds. ¿Hay diferencia? ¿Por qué crees que es pequeña en este caso?
4. *Guardar el Pipeline entrenado* con *joblib* en un archivo **.pkl**. Cárgalo y úsalo para predecir la especie de esta nueva flor: **[5.1, 3.5, 1.4, 0.2]**.

Restricción ética y responsabilidad:

En una celda Markdown responde: el dataset Iris fue creado en 1936 por Ronald Fisher, quien también fue un conocido eugenista.

- ¿Cambia esto algo sobre su uso en educación?
- ¿Qué criterios usarías para decidir si un dataset es éticamente aceptable para entrenar un modelo real?

Además, define por escrito un criterio de 2-3 líneas ("checklist de procedencia de datos") que aplicarías antes de usar cualquier dataset en un proyecto real. Aplica ese criterio al propio dataset Iris y a uno de los datasets que ya usamos en el bootcamp (Titanic o Heart Disease UCI). ¿Alguno de los dos fallaría su propio criterio? Justifica la respuesta.